📖 Manejo de Estado Básico en Flutter

1. **Definición**

En Flutter, el **estado** representa los **datos dinámicos** que pueden cambiar durante la ejecución de la aplicación y que afectan la interfaz de usuario.

- **StatelessWidget**: No tiene estado interno, siempre muestra lo mismo.
- **StatefulWidget**: Puede almacenar y actualizar estado interno, permitiendo que la UI se **reconstruya** cuando los datos cambian.

El mecanismo clave es `setState()`, que notifica a Flutter que el estado cambió y que debe **redibujar la interfaz**.

---

2. **Ciclo de vida de un StatefulWidget**

Un StatefulWidget tiene dos clases:

- La clase principal (`StatefulWidget`) → define la estructura.
- La clase de estado (`State<T>`) → contiene los datos y lógica.

Métodos importantes:

1. `initState()`
   - Se ejecuta una sola vez cuando el widget se crea.
   - Ideal para inicializar variables, cargar datos iniciales o configurar listeners.
2. `build()`
   - Se ejecuta cada vez que se necesita redibujar la UI.
   - Aquí se construye la interfaz visual.
3. `setState()`
   - Se usa para actualizar el estado.
   - Llama nuevamente a `build()` para reflejar los cambios en la UI.
4. `dispose()`
   - Se ejecuta cuando el widget se elimina del árbol.
   - Ideal para liberar recursos (ejemplo: cerrar streams, controladores).

---

3. **Escenarios donde aplicar StatefulWidget**

- **Interactividad simple**: botones que cambian valores, switches, toggles.
- **Contadores**: incrementar/decrementar valores.

- **Formularios**: validar y actualizar campos dinámicamente.
- **Animaciones básicas**: cambiar color, tamaño o estilo de un widget.
- **Estados temporales**: mostrar/ocultar elementos, manejar pestañas.

---

4. **Cómo aplicarse**

1. Crear una clase que extienda `StatefulWidget`.
2. Crear la clase de estado que extienda `State<MiWidget>`.
3. Definir variables de estado dentro de la clase de estado.
4. Usar `setState()` para actualizar esas variables.
5. Construir la UI en el método `build()`.

💻 Ejemplo completo: Contador básico

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Estado Básico',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const CounterScreen(),
    );
  }
}

class CounterScreen extends StatefulWidget {
  const CounterScreen({super.key});

  @override
  State<CounterScreen> createState() => _CounterScreenState();
}

class _CounterScreenState extends State<CounterScreen> {
  int _counter = 0; // Estado interno

  @override
  void initState() {
    super.initState();
    // Inicialización del estado
    _counter = 0;
```

```
  }

  void _incrementCounter() {
    setState(() {
      _counter++; // Actualizamos el estado
    });
  }

  void _decrementCounter() {
    setState(() {
      _counter--; // Actualizamos el estado
    });
  }

  @override
  void dispose() {
    // Liberar recursos si fuera necesario
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Contador con StatefulWidget')),
      body: Center(
        child: Text(
          'Valor actual: $_counter',
          style: const TextStyle(fontSize: 24),
        ),
      ),
      floatingActionButton: Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          FloatingActionButton(
            heroTag: 'decrement',
            onPressed: _decrementCounter,
            child: const Icon(Icons.remove),
          ),
          const SizedBox(width: 20),
          FloatingActionButton(
            heroTag: 'increment',
            onPressed: _incrementCounter,
            child: const Icon(Icons.add),
          ),
        ],
      ),
    );
  }
}
```